



Virtual Ambulance Emergency Response Simulation

Team:

- Lubna Ibrahim Ahmed - 1240350
- Farah Mohamed Ashry - 1240346
- Amena Tamer Elekhtyar - 1240028
- Mariam Amr - 1240266
- Nourhan Karim Adel - 1240274

Cairo University – Faculty of Engineering Credit Hours System – Spring 2026

Course: SBES140 - Computer Graphics & Visualization

Instructor:

Dr. Mohamed Rushdy

Eng. Yara Sayed

Repository: github.com/Lubna-ibrahim-ahmed/Virtual-Ambulance-Emergency-Response-Simulation-in-Blender

Table of Contents

.....	1
1. Introduction & Objectives	3
2. Project Architecture	3
3. The Environment	4
4. Lighting Design	4
Directional light	4
Spotlights	4
Point lights	4
Ambient light.....	4
5. The Scenarios	4
5.1 Scenario 1 — Gas Line Explosion	4
5.2 Scenario 2 — Car Collision	5
5.3 Scenario 4 — Falling Tree (Night Storm)	5
5.4 The Stretcher Rescue Scene	5
6. The Rescue Sequence.....	5
7. Character Animation	6
8. Physics & Collision Detection	6
9. Team Workflow & Version Control.....	7
10. Tools & AI Usage Disclosure	7
11. Testing, Challenges & Lessons Learned.....	8

1. Introduction & Objectives

This project is an interactive three-dimensional emergency-response simulation developed in Unity 6 using the Universal Render Pipeline (URP). The simulation presents a shared low-poly city environment in which multiple accident scenarios unfold, a gas line explosion, a vehicle-pedestrian collision, and a storm-felled tree, each resolving through a complete, unified ambulance response: a paramedic arrives on scene, performs CPR, and evacuates the patient by stretcher.

The educational motivation is twofold. First, emergency-response simulation is a genuinely useful application domain: visualizing the chain from accident to evacuation, hazard, victim, bystander response, emergency call, dispatch, treatment, transport, communicates the structure of real emergency response in a way static material cannot. Second, the project serves as a comprehensive exercise in real-time 3D development, deliberately touching every core area of the course: scene construction and environment design; lighting with all four source types; physics simulation including joint constraints and impulse dynamics; collision detection driving gameplay events; humanoid character animation with retargeted motion capture clips; event-driven software architecture; and interactive user interface design.

Our central technical objective was reusability: rather than building four isolated scenes, we designed one reusable rescue system that responds identically to any emergency, in any scene, without modification. Achieving this shaped the project's architecture and is, we believe, its strongest engineering result.

2. Project Architecture

The project follows an event-driven, decoupled architecture built around ScriptableObject event channels.

Each scenario is orchestrated by a director script that advances the scene through its beats, the hazard event, the victim falling, the bystander's reaction, the emergency phone call. When the call completes, the scenario raises a single shared event, `RescueRequested`, carrying one piece of data: the victim's world position. The scenario neither knows nor cares what listens to that event.

The `RescueSequence` prefab is the listener. Dropped into any scene, it subscribes to `RescueRequested` and, upon receiving it, executes the full response: the ambulance drives in with flashing beacons and siren, the paramedic exits and walks to the patient, performs CPR, transfers the patient to a stretcher, loads, and departs. On completion it raises a second channel event, `RescueCompleted`, which triggers the scenario's ending treatment (an end card with a return-to-menu option).

This decoupling produced three concrete benefits. First, the rescue was written once and reused in every scenario, four emergencies, one response system. Second, team members developed scenarios in parallel on separate machines without touching shared code: each scenario only needed to raise the channel event. Third, positions are derived at runtime from the event payload, the rescue computes the paramedic's kneeling point, the CPR position, and the stretcher placement from the patient's actual transform, so the same prefab adapts to any scene layout.

Events are guarded to fire exactly once (once-only flags on both the request and completion), a discipline adopted after early testing exposed repeated-event bugs.

3. The Environment

All scenarios share one city environment built from a low-poly asset pack: streets with sidewalks and curbs, multi-story buildings, streetlamps, trees, traffic infrastructure, and parked vehicles. The shared environment reinforces the simulation's coherence, the same neighborhood experiences each emergency, and allowed lighting and navigation work to transfer between scenarios.

A deliberate material distinction runs through the project: the environment uses flat-color materials (solid albedo plus URP Lit shader properties, no image textures), giving the clean low-poly aesthetic, while characters carry full image texture maps UV-mapped to their meshes. This contrast is visible on inspection and was a conscious stylistic and performance choice.

4. Lighting Design

Every scenario scene implements all four light source types, tuned to its time of day:

Directional light the sun or moon: one per scene, global, parallel rays where only rotation matters. Scenario 4 uses a dim, blue-tinted directional as storm moonlight; Scenario 2 a bright warm directional as midday sun.

Spotlights cone-shaped sources placed at justified fixtures: streetlamp heads casting pools onto the sidewalk, shop and awning lights. Justification the light visibly originating from a believable source was a design rule throughout.

Point lights omnidirectional radiators: shop entrance bulbs, and most prominently the ambulance's flashing red/blue beacons, implemented as small-range point lights so they produce an intense local glow without contaminating scene-wide lighting.

Ambient light configured per scene in Unity's Environment Lighting settings: a dark blue ambient fills shadows in the night scenes so unlit surfaces never render pure black; the day scene uses bright neutral ambient.

When light strikes a surface, the shader combines three responses per pixel: diffuse reflection (light scattered evenly in all directions — the object's perceived color, viewing-angle independent), specular reflection (the mirror-like highlight concentrated by surface smoothness, which shifts with viewing angle), and the ambient term (indirect fill approximating bounced environmental light). Material smoothness values control the diffuse/specular balance across the project's surfaces.

All lighting is real-time rather than baked. Baked lighting pre-computes static light contribution into lightmap textures; excellent for performance on static scenes, but incompatible with this project's requirements: the ambulance beacons flash dynamically, characters and physics objects move through the light, and our scenarios depend on dynamic shadows. In a production setting a hybrid approach (baked static architecture, real-time dynamic lights) would be the optimization path.

5. The Scenarios

5.1 Scenario 1 — Gas Line Explosion

The scenario is player-triggered: an on-screen prompt instructs "Press X to trigger the gas leak," and the explosion with its visual effects and sound fires on the keypress. This scenario demonstrates interactive event triggering and audio-visual effect synchronization. During integration testing we diagnosed and repaired a sound-timing defect: the explosion audio was being played by a stray

camera AudioSource with Play On Awake enabled (firing at scene load), while the controller's intended source pointed at a disabled object; the fix re-pointed the controller to the active source so the sound fires exactly once, on X. The emergency-response chain (witness, call, rescue) completes the scenario through the shared architecture.

5.2 Scenario 2 — Car Collision

A daytime scenario contrasting the night scenes. Kate walks along the sidewalk toward a crossing while background pedestrians populate the street; the player starts the vehicle ("Press D") — the car then approaches in parallel with her walk, visible in the establishing shot, building anticipation. As she steps into the lane, the car enters a brake-trigger zone: it decelerates with a tire-screach but strikes her. The impact fires three simultaneous consequences: Kate's fall animation ending grounded on the asphalt; a physics debris burst Rigidbody fragments shaped as car parts (panels, a hubcap, body-colored chunks) switched from kinematic to dynamic and scattered by impulse forces, bouncing and rolling under full physics simulation; and a clinical blood decal static dark circular pools revealed beneath her, deliberately desaturated and restrained in keeping with a medical simulation. The nearest pedestrian becomes the witness: shock reaction, then a phone call (with a phone prop in hand) that raises RescueRequested.

5.3 Scenario 4 — Falling Tree (Night Storm)

The night scenario: rain, thunder, wind, and storm lightning. Kate walks beneath a large tree as lightning strikes it, the only scenario with an automatic trigger, a deliberate design statement: human and mechanical hazards are player-triggered, but nature strikes on its own. The tree's fall is the project's centerpiece physics demonstration (§8): a Rigidbody constrained by a HingeJoint, released with applied torque, felled by gravity. The canopy strikes Kate (detected by a trigger zone); she falls and lies on the street beside the foliage; a witness reacts, makes the 911 call with the phone prop, and the rescue chain begins.

5.4 The Stretcher Rescue Scene

A dedicated scene used to develop and showcase the rescue system in isolation, the rescue equivalent of a test stage. It remains in the project as the development environment in which the rescue sequence was built and verified before integration into the scenarios.

6. The Rescue Sequence

The shared response system, packaged as a single prefab, comprising:

The ambulance driving along waypoint approach paths with displacement-driven wheel rotation (wheels spin proportionally to movement, so the vehicle reads as driving rather than sliding); a light bar of alternating red/blue point lights with emissive material pulses; and a procedurally synthesized two-tone siren generated in code via AudioClip.Create no audio file exists; the wail is computed at runtime (a self-made asset).

The paramedic (Chad) a Mixamo character in medical scrubs, animated with dedicated clips: Administering CPR (looping chest compressions), Tender Placement (the kneel-and-lift), and Being Carried (the patient's supine pose). His positioning is body-frame derived: rather than hard-coded coordinates, the system computes a frame from the patient's actual transform her chest bone, body axis, and the road-side perpendicular and derives his kneeling point (≈ 0.55 m beside her chest, rotated to face her), his walking paths (routed around her body with ≥ 1 m clearance on every

segment, treating her as a 2 m × 0.6 m ground obstacle), and the stretcher's stopping position from it. This made the rescue robust to any victim position in any scene.

The stretcher modeled from Unity primitives (frame rails, mattress, four-wheel cylinders a self-made asset) with the same displacement-driven wheel spin. Patient placement computes the mattress surface height from the mesh's top bound and raises the patient so her back rests exactly on the surface measured, not guessed. A defibrillator prop rides on the stretcher's foot end, arriving with the response equipment.

Interactivity the rescue contains the simulation's medical decision points as keyboard interactions with on-screen prompts: E dispatches the ambulance after the 911 call; C begins CPR when the paramedic reaches the patient; F initiates the lift onto the stretcher. The player acts as dispatcher and paramedic.

Completion the ambulance departs along an exit path leaving the frame; RescueCompleted is raised exactly once; after a beat, an end card fades in "Patient Evacuated Scenario Complete" with a return-to-menu option.

7. Character Animation

All characters originate from Adobe Mixamo and were converted to Unity's Humanoid rig a standardized skeletal mapping that enables retargeting: a single animation clip drives any humanoid character. One walk cycle animates Kate, the paramedic, and every background pedestrian; the fall, shock, phone-call, CPR, and placement clips likewise retarget to whichever character needs them.

Mechanically, each character mesh is bound to a skeleton a hierarchy of bones from hips through spine to extremities by skinning: every mesh vertex is weighted to nearby bones, so bone rotations deform the mesh naturally. Animation clips record bone rotations over time. The skeleton/animation system is distinct from the physics system: an Animator drives bone poses; a Rigidbody drives whole-object physics. Our characters are animation-driven; our tree is physics-driven.

Each character carries an Animator Controller a state machine governing clip playback: Idle and Walk states transition on an IsWalking parameter synchronized with waypoint locomotion (characters translate along waypoints while playing the walk cycle both are required, or characters glide); one-shot actions (fall, shock, phone call, CPR triggers) fire on Animator triggers. A project-wide discipline emerged on loop flags: looping clips (walk, idle, CPR compressions) have Loop Time enabled, one-shot clips have it disabled a lesson learned when a looping phone-call clip caused an endless call.

Character textures passed through a Blender pipeline: several Mixamo characters imported with unlinked textures (rendering gray).

8. Physics & Collision Detection

Rigidbody dynamics. A Rigidbody places an object under the physics engine's control. Unity distinguishes dynamic bodies (governed by gravity, forces, and collisions) from kinematic bodies (governed by scripts/animation; immune to forces but able to affect dynamic bodies). The project uses both and transitions between them deliberately: the falling tree is dynamic during its fall and switches to kinematic upon settling, freezing it stably against post-impact jitter.

The tree the centerpiece: a Rigidbody (mass 300) constrained by a HingeJoint at its base a joint permitting rotation around a single axis, like a door hinge. The fall is initiated by a single applied

torque; gravity completes it. No animation is involved: the fall is genuine simulation, evidenced by slight run-to-run variation. Settling is contact-based rather than timed an earlier timer-based implementation froze the tree mid-air when the fall outlasted the timer, a bug whose fix (settle on ground contact) illustrates why event-driven completion beats timed assumptions (§11).

The debris Rigidbody fragments held kinematic until the car impact, then switched dynamic and scattered by AddForce impulses, bouncing and rolling wherever the simulation takes them.

Collision detection the physics engine tests collider overlaps each step and reports them to scripts. The project uses both collider roles: solid colliders (capsules on characters, boxes on environment and vehicles) prevent interpenetration; trigger colliders detect without blocking, and drive the scenarios' key events the car's impact zone striking Kate, the tree canopy's contact zone, the vehicle's brake zone each firing OnTriggerEnter handlers that advance the scenario exactly once.

9. Team Workflow & Version Control

The project is versioned with Git and GitHub, with Git LFS (Large File Storage) handling the binary assets FBX models, textures, and audio (65 LFS-tracked files, ≈930 MB) keeping the repository clonable while preserving full asset history.

Development ran in parallel across multiple machines (Windows and macOS) and team members, coordinated by a one-owner-per-scene discipline: each scenario scene had a single owner at any time, with a pull-before-work, push-after-verify rhythm. The shared event-channel architecture made this practical scenarios integrate through one event, not through shared code, so parallel work rarely touched the same files. The repository contains the complete Unity project, course assignments, and documentation (README), per submission requirements.

1. Virtual Environment Creation (15%) Nourhan
2. Hero Object Creation (15%) Mariam, Amena, Farah
3. Character Modeling and Skeletal Animation (15%) Lubna
4. Emergency Response Scenarios (15%) Mariam, Amena, Farah
5. Lighting and Rendering (10%) Lubna
6. Object Interaction and Collision Handling (15%) Mariam, Nourhan
7. Physics-Based Rendering Component (15%) Amena, Farah
8. Optional Interactive Extension (10% BONUS) Farah

Mariam: 5% Hero, 5% Scenario, 10% Object Interaction

Farah: 5% Hero, 5% Scenario, 5% Physics, 10% Bonus

Lubna: 15% Characters, 10% Lighting,

Amena: 5% Hero, 5% Scenario, 10% Physics-Based Rendering Component

Nourhan: 15% Environment, 5% Object Interaction

10. Tools & AI Usage Disclosure

In accordance with course policy, all tools used are disclosed below:

ToolPurposeUnity 6 (URP)Game engine: scenes, lighting, physics, animation systems

UIBlenderCharacter texture-extraction pipeline

Adobe MixamoCharacter models and animation clips

City environment modelsClaude (Claude Code)Assistive tool: C# code suggestions and script drafting for scenario logic and the rescue sequence, under the team's direction. All scenario design, scene

construction, physics tuning, testing, debugging, verification, and integration were performed by the team, and all generated code was reviewed, corrected, and validated by the team through repeated play-testing. Git, GitHub, Git LFS Version control and team coordination

11. Testing, Challenges & Lessons Learned

Methodology. Every feature was verified through repeated Play-mode runs against explicit criteria: events fire exactly once; characters end grounded on surfaces; physics behaves consistently; full chains complete without console errors. Verification was visual as well as logical screenshots captured per beat, with close-ups for texture and grounding checks, after learning that distant screenshots could pass while close inspection failed.

Challenges solved (selected):

- The looping phone call. The witness's call never ended: the one-shot clip had Loop Time enabled, and the call-finished event re-fired every loop. The fix established two project-wide disciplines: loop flags set deliberately per clip, and once-only guards on every consequential event.
- The floating patient. Retargeted animation end-poses left Kate suspended ~0.5 m above the street. Grounding became an explicit verification criterion, with end positions corrected against the actual road surface.
- The mid-air tree. A timer-based settle froze the tree (switched it kinematic) before it finished falling, pinning it in the air. The fix settling on ground contact rather than a timer became an architectural principle: event-driven completion over timed assumptions.
- Spatial correctness in the rescue. Early rescue runs exposed a class of positioning bugs: the paramedic's path crossed the patient's body; CPR played beside rather than over her; the patient clipped into the stretcher mattress. The systematic fix replaced hard-coded positions with body-frame-derived computation (positions calculated from the patient's actual bones and body axis, paths routed around her with clearance) and measured surface placement (mattress height read from mesh bounds). The lesson: in dynamic scenes, derive positions from runtime state; never hard-code.
- Multi-machine coordination. Establishing the Git LFS pipeline across machines and team members including large-binary handling and scene-ownership discipline was itself an engineering effort, repaid in conflict-free parallel development.

Conclusion

The project met its objectives: playable emergency scenarios unified by a single reusable response system, demonstrating environment and lighting design, genuine physics simulation, retargeted character animation, event-driven architecture, and interactive UI built collaboratively under version control, tested visually and logically, and documented honestly. The strongest engineering outcomes were the decoupled rescue architecture and the debugging discipline that emerged from real failures: fire events once, ground every pose, complete on contact, and derive from runtime state.